

Universidad Nacional Autónoma de México

Facultad de Ingeniería



Sistemas Distribuidos

Semestre 2027 - 1

Examen escrito en casa

Alumno: Fernández Herrera Mauricio

Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

Entrega: 21/11/2026

Sistemas Distribuidos · 2027 - 1 Entrega: 09/11/2026
Fernández Herrera Mauricio.

Examen Escrito en Casa.

• Preguntas Fijas.

- 1.7: ¿Qué diferencia hay entre un monolito y microservicios, y cuándo usarías cada uno?

Un monolito es una aplicación donde sus funcionalidades están integradas en un solo proyecto y se despliegan juntas. Los microservicios dividen la aplicación en servicios independientes que se comunican entre sí. Usaría un monolito para proyectos pequeños porque es más sencillo de desarrollar, probar y desplegar; eligiría microservicios cuando el sistema crece, ya que permite escalar cada servicio por separado y de manera diferente.

- 2.4 ¿Qué significa que una operación sea idempotente, y cómo harías que reintentar un cobro no cobre dos veces?

Una operación es idempotente cuando ejecutarla varias veces produce el mismo resultado que ejecutarla una sola vez. Para evitar cobrar dos veces usaría una clave de idempotencia única por cada intento de pago y la enviaría junto con la solicitud al mecanismo de cobro. Gano seguridad ante reintentos, pero la necesidad de manejar y almacenar dichas claves.

- 2.5 ¿Qué pasa si la llamada de tu servicio a una pasarela de cobro vence su timeout? ¿Qué sabes y qué no sabes en ese momento?

Un timeout significa que mi servicio no recibió una ...

... respuesta en el tiempo esperado. Sé que no recibí confirmación, pero no sé si la pasarela procesó el cobro, si sigue trabajando o si nunca recibió la solicitud. Reintentar sin control podría generar algún cobro duplicado.

- 3.6 ¿Qué diferencia hay entre un API Gateway y un balanceador de carga, y cuándo pondrías cada uno?

Un balanceador de carga distribuye las solicitudes entre varias instancias de un servicio. Un API Gateway funciona como un punto de entrada para varios servicios y puede encargarse de autenticación, autorización, rutas, límites y solicitudes. Usaría el balanceador para repartir tráfico entre réplicas ya que no entiende la lógica de negocio, y una API Gateway cuando tenga varios servicios que necesiten una entrada común, aunque esto significa verificar que ese otro componente se mantenga disponible.

- 4.1 Explica el teorema CAP y qué implica para el diseño de un sistema de datos replicados.

El teorema CAP establece que, ante una partición de red, un sistema debe elegir entre consistencia y disponibilidad. Todos los nodos conservan los mismos datos actualizados, o se busca que cada solicitud reciba una respuesta. Para un sistema con datos replicados elegiría priorizar consistencia al tratarse de datos bancarios, y disponibilidad con algo como un catálogo.

- 5.3 ¿Qué diferencia hay entre una Saga y el commit en dos fases, y cuándo elegirías cada uno para una operación que cruza varias bases?

Una Saga divide una operación distribuida en varias transacciones locales y, si alguna falla, ejecuta acciones a modo de compensación para revertir los efectos anteriores.

El commit en dos fases coordina a las bases para que todos confirmen o cancelen una transacción, evitando resultados parciales. Usaría Saga para procesos largos como realizar un pedido, y 2PC cuando sea indispensable que varias bases confirmen juntos y soporten ese protocolo

- 6.3. Cola a punto, pub/sub o log: ¿Qué diferencia hay entre los tres y cuándo elegiría cada uno?

Una cola punto a punto entrega cada mensaje a un solo consumidor, por lo que sirve para repartir trabajo entre varios trabajadores. Pub/sub permite que varios suscriptores reciban el mismo evento, cada uno de forma independiente. Un log almacena los mensajes en orden y permite que los consumidores los lean desde una posición y los vuelvan a consultar. Usaría una cola para procesar pedidos, pub/sub para notificar eventos y un log para registrar movimientos que después se necesiten consultar o reproducir.

- 6.4 Explica por qué broker no puede prometerte que cada mensaje se entregue exactamente una vez y qué puedes prometer en su lugar.

Broker no puede garantizar esto porque puede ocurrir una falla después de procesarlo, pero antes de confirmar su recepción. En ese caso, el broker no sabe si eliminarlo o volver a enviarlo, por lo que puede entregarse duplicado. Para manejarlos, el consumidor debe ser idempotente y reconocer mensajes mediante un identificador único, así se puede garantizar que los efectos de un mensaje se apliquen una sola vez, aunque físicamente se reciba varias veces.

- 7.2. Explica la relación "Sucedió antes" y como la respeta un reloj de Lamport.

La relación "Sucedió antes" establece un orden causal entre eventos. Si un evento provoca otro, entonces ocurrió antes que él. El reloj Lamport asigna un contador lógico a cada evento, incrementándolo cuando ocurre uno nuevo. Al enviar un mensaje, se incluye al contador y quién lo recibe actualiza su reloj al máximo entre su valor y el recibido, más uno. Esto garantiza que si un evento ocurrió antes que otro, su marca de tiempo también será menor. Sirve para ordenar los eventos en sistemas distribuidos, aunque no sabe distinguir si dos eventos fueron simultáneos o independientes.

- 8.4: ¿Cuándo pondrías una caché frente a tu base de datos y que no cachearías nunca?

Pondría una caché frente a la base de datos cuando existen datos que se consultan muchas veces y cambian poco, para responder más rápido y reducir la carga de la base. A cambio, necesitaré administrar la expiración y actualización de los datos, porque la caché puede contener información desactualizada. No usaría la caché como mi única fuente de la verdad para datos críticos, sino que esos datos deben ser consultados en la base de datos para evitar decisiones basadas en información incorrecta.

- Preguntas Adicionales.

1.3: Explica que es la elasticidad y en que se diferencia de la escalabilidad.

La escalabilidad es la capacidad de un sistema para manejar más carga al agregarle recursos sin rediseñarlo. La elasticidad es la capacidad de hacerlo de forma...

... automática y en ambos sentidos: sumar recursos cuando sube la demanda y quitarlos cuando baja. Con elasticidad gano que pago solo lo que uso y que los picos se absorben sin intervención humana, pero pierdo previsibilidad en el coste y asumo complejidad extra.

2.1. ¿Qué diferencia hay entre REST y gRPC y cuándo elegirías gRPC?

Rest expone recursos sobre HTTP, normalmente con JSON en texto, así que es legible, cacheable y lo puede consumir cualquier cliente. gRPC permite llamar funciones remotas definidas en un contrato Protocol Buffers, viaja sobre HTTP/2 en formato binario y genera código para cliente y servidor. Gano velocidad, streaming, mensajes más pequeños, pero pierdo legibilidad y facilidad para depurar y/o cachear. Elegiría gRPC para la comunicación interna entre servicios con mucho tráfico y baja latencia, y dejaría REST para una API pública que consumen clientes que no controlo.

2.8. Explica el modelo de madurez de Richardson y qué gana una API cuando usa los verbos y códigos HTTP según su significado.

El modelo evalúa qué tan RESTful es una API en 4 niveles. En el nivel 0 hay un solo endpoint y un solo verbo, usando HTTP como túnel. En el nivel 1 aparecen recursos con su propia URL. En el nivel 2 se usan los verbos y códigos de estado con su significado. En el nivel 3, las respuestas incluyen enlaces que indican qué se puede hacer a continuación.

2.10. ¿Qué diferencia hay entre que dos servicios se comuniquen de forma síncrona o asíncrona y cuándo eligirías cada una?

En la comunicación Síncrona el Servicio que llama envía la petición y espera la respuesta antes de seguir. En la asíncrona manda el mensaje a una cola o broker y continúa, y el otro Servicio la procesa cuando pueda. Usaría Síncrono para operaciones como consultar el saldo de una cuenta, y asíncrono para enviar el correo de confirmación después de un pedido.

3.10. ¿Qué diferencia hay entre repartir carga por turnos (Round Robin) y por menos conexiones, y cuándo elegirías cada una?

Round Robin manda cada petición al siguiente Servidor en orden circular, sin mirar su estado. Con menos conexiones la manda al Servidor que tenga menos conexiones en ese momento. Round robin es simple, barato y funciona bien si los Servidores son muy similares, pero se descompone cuando algunas peticiones son largas, porque un Servidor se satura mientras otro está disponible. Menos conexiones se adapta a la carga real, pero obliga al balanceador a llevar la cuenta. Usaría round Robin para una API con peticiones cortas y uniformes, y menos conexiones para sesiones largas o de duración desigual.

3.14. ¿Qué diferencia hay entre replicar y particionar los datos, y cuándo usarías cada uno?

Replicar es guardar copias completas de los mismos datos en varios nodos. Particionar (sharding) es dividir el conjunto de datos en fragmentos y que cada nodo guarde solo uno. Usaría replicación cuando el conjunto cabe en un nodo y lo que importe sea la disponibilidad y tolerancia a fallos, y partición cuando los datos o la carga de escritura excedan una sola máquina, aunque me arriesgo a perder fragmentos de información

4.5 ¿Qué diferencia hay entre replicación síncrona y asíncrona, y cuándo elegirías cada una?

En la síncrona, el líder no confirma la escritura al cliente hasta que las réplicas la han recibido, no se pierden datos confirmados pero pagamos latencia. En la asíncrona, el líder confirma de inmediato y propaga después, es rápida y tolerante a réplicas lentas, pero si el líder cae podemos perder escrituras recientes. Usaría síncrona cuando perder un dato es inaceptable y asíncrona cuando importa más el rendimiento o réplicas lejanas.

4.10: Cómo elige Raft a un líder, y por qué no puede haber dos líderes en el mismo término?

Cada seguidor tiene un temporizador de elección con valor aleatorio; si no recibe latidos del líder antes de que expire, se convierte en candidato, incrementa su término, vota por sí mismo y pide votos. Cada nodo vota como máximo una vez por término. Quien reúne la mayoría, gana. No puede haber dos líderes porque dos mayorías cualesquiera de un mismo clúster comparten al menos un nodo, y ese nodo solo votó una vez en ese término. La aleatoriedad de los temporizadores reduce los empates; si ocurren, se inicia otro término.

4.12 ¿Raft tolera más fallas que Paxos o es más rápido? ¿Qué gana entonces?

Ambos toleran f fallas con 2f+1 nodos porque dependen de mayorías, y en estado estable su rendimiento es comparable. Lo que gana Raft es comprensibilidad e implementabilidad: descompone el problema en elección de líder, replicación del log y seguridad, impone un líder fuerte y un log sin huecos.

5.1. Explica qué garantiza una transacción ACID y por qué esa garantía se pierde cuando una operación cruza varios servicios, cada uno con su base.

Una transacción ACID garantiza atomicidad, consistencia, aislamiento entre ellas y durabilidad. Cuando una operación cruza varios servicios cada base confirma por su cuenta, así que un servicio puede hacer commit y otro quedar a medias, y nada impide que otros lean estados intermedios.

6.1. ¿Cuándo pondrías una cola de mensajes entre dos servicios y cuándo no?

Pondría una cola cuando el productor no necesita la respuesta para continuar y quiero desacoplar a los servicios en el tiempo: absorber picos de carga, tolerar que el consumidor esté caído o lento, repartir trabajo entre varios consumidores o notificar un mismo evento a varios interesados. No la pondría cuando el llamador necesita el resultado inmediatamente

6.13 ¿Qué pasa si un consumidor se cae a la mitad de procesar un mensaje y para qué sirve una cola de mensajes muertos? (DLQ)

Si el consumidor confirma solo después de procesar, un fallo a mitad de camino deja el mensaje sin confirmar, la cola vence su plazo de visibilidad o oculto la desconexión y lo reentrega a otro consumidor. Si se confirmara antes de procesar un fallo perdería el mensaje. La cola de mensajes muertos sirve para los mensajes que fallan una y otra vez, ya sea por datos mal formados o por un error persistente, se aíslan de la cola principal para inspeccionarlos, corregirlos y reprocesarlos.

7.5. ¿Qué diferencia hay entre un reloj de Lamport y uno vectorial, y cuándo pagarías el coste del vectorial?

Ambos ordenan eventos. El de lamport usa un solo cont... ..dor por proceso, se incrementa en cada evento y al recibir un mensaje, se toma el máximo entre el propio y el del mensaje más uno. El reloj vectorial mantiene un contador por proceso y se combina tomando el máximo componente a componente comparando dos vectores. Se pagaría su coste cuando se necesite detectar conflictos reales, por ejemplo en bases replicadas con escrituras en varios nodos.

8.3. Explica qué es la persistencia políglota y qué te cuesta usar varias bases de datos a la vez.

Es la práctica de usar varios tipos de bases de datos dentro de un mismo sistema, eligiendo para cada necesidad la que mejor encaja. Lo que cuesta es la complejidad, aparte de las distintas tecnologías que se deben manejar, los datos se duplican entre almacenes, por lo que deben mantenerse sincronizados. Además se pierden transacciones ACID entre las bases.

9.1. ¿Cuándo necesitas una blockchain y cuándo te basta una base de datos?

Una blockchain tiene sentido cuando varias partes que no confían entre sí necesitan compartir un registro, ninguna debe controlarlo y no hay un tercero aceptado por todas que pueda administrarlo, y además necesitan que el historial sea inmutable y verificable. Se paga bajo rendimiento, latencia de confirmación, coste de consenso y dificultad para corregir errores o cumplir normativas como el derecho al borrado de datos.